Architectural Enhancement Report for Gemini CLI

Group 5: It Compiles (Sometimes)

Assignment 3

Monday, April 6, 2026

Group Leader: Edward Tanurkov - edward.tanurkov@queensu.ca

Presenter 1: Brian Constandes - 22bac1@queensu.ca

Presenter 2: Arham Hassan - 23cr8@queensu.ca

Ajwaad Khan - 19aak19@queensu.ca

Junoh Loy - 22DLT1@queensu.ca

Faizan Muhammad - 22mq34@queensu.ca

Table of Contents

1 Abstract

2 Introduction

3 Enhancement Motivations and Benefits

4 Architectural Impact

4.1 High-Level Conceptual Architecture

4.2 Low-Level (Subsystem) Modifications

5 Use Cases and Sequence Diagrams

5.1 Use Case 1

5.2 Use Case 2

6 Implementation Alternatives

6.1 First Alternative - Centralized Tagging and Policy Layer

6.2 Second Alternative - Distributed Tag-Aware Enhancement

7 Effects on System Quality Attributes

7.1 Maintainability

7.2 Evolvability

7.3 Testability

7.4 Performance

8 SEI SAAM Analysis

8.1 Stakeholders

8.1.1 Security Engineers

8.1.2 Enterprise Administrators

8.1.3 MCP Developers

8.1.4 General End Users

8.2 NFRs

8.3 Evaluation

8.4 Recommendation

9 Interactions with Existing Features

10 Testing Plan

11 Risks

12 Lessons Learned

13 Conclusion

14 AI Collaboration Report

14.1 Addition Brainstorming

14.2 Planning Document Outline

14.3 AI Collaboration Methodology

15 References

1 Abstract

Our report proposed an architectural enhancement to Gemini CLI that introduces a content-tagging and policy-enforcement layer to address semantic security vulnerabilities and prompt injection. Although classifiers and sandboxing are current defenses, the system does not have consistent tracking of content origin and trust levels throughout the pipeline.

Our enhancement recommendations add a ContentTaggingService and PolicyEngine to assign provenance-based metadata (e.g., trusted, untrusted, external) and enforce context-aware security policies. This enables stricter handling of untrusted inputs while preserving performance for trusted workflows.

The design integrates with existing components such as the InjectionService and SandboxManager without modifying the core behaviour of the LLM. The system is able to achieve stronger and more consistent security while maintaining modularity and scalability through improved input separation and provenance tracking.

2 Introduction

Gemini CLI is a powerful agentic development tool which integrates LLMs with file operations, web access, and execution capabilities. However, it is vulnerable to certain things like prompt injection and manipulation through untrusted external content, like many other LLM-based systems.

Although existing mechanisms such as injection classifiers and sandboxing provide standard protection, they do not operate with a consistent notion of content rust or origin. As a result, malicious or external inputs may be treated similarly to trusted system instructions, creating a structural security gap.

Our report proposes a content tagging and policy enforcement layer which introduces provenance tracking and trust-aware processing into the system pipeline. When we explicitly distinguish between trusted and untrusted content, the enhancement enables more robust, context-sensitive security decisions while preserving usability and system performance.

3 Enhancement Motivations and Benefits

Gemini CLI is architecturally defined by its agentic capabilities, meaning it can act independently to perform tasks. Specifically, it can automatically read and write files, execute shell commands (commands run through the computer's terminal), fetch web content from the internet, and connect to external services using the Model Communication Protocol (MCP), which is a framework for communication between models and other systems. This agentic nature gives the tool power. However, it also causes a structural security exposure that the current architecture does not fully address.

The Structural Gap. The A2 concrete architecture (Section 4.5) shows Gemini CLI's Safety and Approval Layer runs as a four-module pipeline [6]. These modules are: packages/core/src/safety/ for content safety checks, meaning it analyzes input to ensure it does not violate safety policies; packages/core/src/policy/ for tool approval rules via PolicyEngine, which is the component that decides if a tool invocation should proceed;

packages/core/src/confirmation-bus/ for routing approval events, acting as the system's internal communication mechanism; and packages/core/src/scheduler/ for orchestrating approved tool calls, which manages the execution order of approved actions. These mechanisms are genuine and functional. However, none have a consistent, pipeline-wide understanding of *content origin*. Once content enters the pipeline—whether from the terminal (user input in a command line), a web fetch (data retrieved from the internet), an MCP tool (outputs from Model Control Plane tools), or a system-generated instruction (operations initiated internally)—it loses its identity. The PolicyEngine in policy-engine.ts checks if a tool should run. However, it does not verify if the content that triggered that decision came from a trusted or untrusted source. This is the gap: **provenance is lost at ingestion**. Content from an external web page and content typed by the developer are architecturally identical downstream.

Why This Matters for Gemini CLI Specifically. Indirect prompt injection in agentic systems is not just theoretical. OWASP's Top 10 for LLM Applications lists prompt injection as the top risk, calling out agents that act on external content [1]. Google DeepMind's research found that undefended agentic systems tested with TAP-based attacks were compromised in almost all cases [2], [3]. Gemini CLI's toolset—web-fetch.ts, web-search.ts, mcp-tool.ts, and read-file.ts—creates the attack surface these studies warn against. In Gemini CLI, agents retrieve external content and act on it within the same reasoning loop. The key architectural fact is that Gemini CLI's existing safety pipeline focuses on tool execution, not content provenance. It asks, "Should this tool run?" but not, "Should content from this source influence this decision?" These are different questions. The second is currently unanswered at the pipeline level.

New Module or Augmented Existing Feature. The proposed enhancement adds a new module: **packages/core/src/tagging/,** with a ContentTaggingService class. This integrates as a **new entry point into the existing pipeline. The ContentTaggingService** is not a standalone system and does not replace any component. It sits just before the safety checks so that tagged content flows through existing checks into a slightly extended PolicyEngine. PolicyEngine is augmented (not replaced) to accept provenance tag metadata for enforcement. The SandboxManager also gains a method to adjust execution constraints at the tag level. Scope is limited: one new directory, two modified methods, and no changes to the Orchestration Engine, Tool Execution Layer, or CLI Frontend.

Tag Design. The ContentTaggingService assigns each content object a structured metadata envelope with three fields. The TRUST_LEVEL field is either TRUSTED, UNTRUSTED, or EXTERNAL. Direct user CLI input is tagged TRUSTED. Content from web-fetch.ts or web-search.ts is tagged EXTERNAL. Content from the MCP tool responses from non-allowlisted servers is tagged UNTRUSTED. The SOURCE_TYPE field is a categorical label: USER_CLI, WEB_FETCH, WEB_SEARCH, MCP_TOOL, or SYSTEM_GENERATED, based on entry point. The RISK_SCORE field is a 0.0–1.0 floating-point heuristic. It uses the source type, input channel trust level, and a simple pattern match to detect known prompt injection signatures.

Storage and Visibility. Tag metadata is only stored **in memory. It is** attached to the content wrapper object as it passes through the pipeline. Tags are never written to disk, sent to the Gemini API, or sent to remote MCP servers. This protects privacy: no provenance information leaves the local process. The storage model remains simple with no new database or filesystem dependencies. Backward compatibility with content interfaces is preserved. Tags are invisible to users by default. They see only effects, such as warnings when content is

blocked or normal output when it isn't. Enterprise administrators in audit mode can receive logged tag summaries from the PolicyEngine. Tags stay inside the local process only.

Benefits by Stakeholder. For security engineers, the enhancement makes trust handling explicit and auditable. Security properties that were implicit become visible and testable through the tag layer. Updates to tagging logic are kept in one component. For enterprise administrators, the enhancement adds configurable trust policies. They can define MCP allowlists, set risk-score thresholds for prompts, and retrieve audit logs without changing pipeline logic. For MCP developers, the tag system provides a predictable contract: allowlisted integrations work without friction. Unregistered ones get UNTRUSTED treatment. For end users, the enhancement protects against indirect prompt injection without extra steps on trusted workflows. Direct CLI commands are tagged TRUSTED and pass through with little overhead.

Novelty Precision. The claim is not that Gemini CLI lacks security. The system's current safety pipeline, confirmed by the A2 concrete architecture, is genuine and functional. The claim is that these defences are not provenance-aware. They cannot distinguish malicious instructions from legitimate user instructions. The ContentTaggingService closes this exact gap. It makes content origin an explicit and enforceable system property. Now, the existing pipeline can act on content based on its origin rather than treating all content equally.

4 Architectural Impact

4.1 High-Level Conceptual Architecture

The existing Gemini CLI system follows a modular, pipeline-based architecture in which user inputs and external content are processed through a series of services, including input handling, the InjectionService, sandboxing mechanisms, and the LLM processing stage. While security measures exist, the system does not consistently track the origin or trust level of content as it flows through the pipeline. This creates a structural vulnerability in LLM-based systems, where inputs coming from external or unverified sources can influence model behaviour without consistent validation or tracking.

Recent research and reports, including OWASP’s designation of prompt injection as a top risk for LLM systems [1], show that traditional defences are insufficient in systems that process natural language and intent. Particularly, the absence of content provenance tracking and input separation allows malicious or manipulated content to be treated equally to trusted system instructions.

To address this, the proposed enhancement introduces a new **Content Tagging and Policy Enforcement Layer** that operates early in the input processing pipeline. This layer consists of a ContentTaggingService, which assigns metadata tags to incoming content based on its origin (e.g., user input, external data source, or system-generated content), and a **Policy Enforcement component**, which ensures that services handle content according to its assigned risk level.

At a high level, when content enters the system, it is first sent through the ContentTaggingService, which analyzes its source and assigns it a tag such as trusted, untrusted, or external. The tagged content is then passed to existing security components. These components then determine how to handle it.

This design builds on the existing architecture without modifying the core LLM behaviour. Instead, it introduces a modular control layer that enhances visibility and enables policy-based decision-making across the pipeline. By maintaining loose coupling between components, the system improves security while preserving scalability and maintainability.

4.2 Low-Level (Subsystem) Modifications

To support the proposed content tagging and policy enforcement, several subsystem-level modifications are required. These changes include new components, updates to existing services, and extensions to system interfaces.

Modified Components

- InjectionService: Extended to interpret content metadata tags and enforce different validation or filtering strategies based on trust level. For example, inputs tagged as “untrusted” will trigger stricter sanitization and injection checks.
- SandboxManager: Modified to adjust execution constraints based on content tags dynamically. High-risk or external content may be executed in more restrictive sandbox environments.
- InputHandler: Updated to ensure that all incoming content is routed through the ContentTaggingService before reaching downstream components.

New Components

- ContentTaggingService: Responsible for analyzing incoming content and assigning metadata tags. It determines the origin of content(e.g., user-provided, retrieved from external sources, or system-generated) and assigns corresponding trust levels.
- PolicyEngine: Applies system-defined security policies based on content tags. It acts as a centralized mechanism to standardize how tagged content is handled across different services.

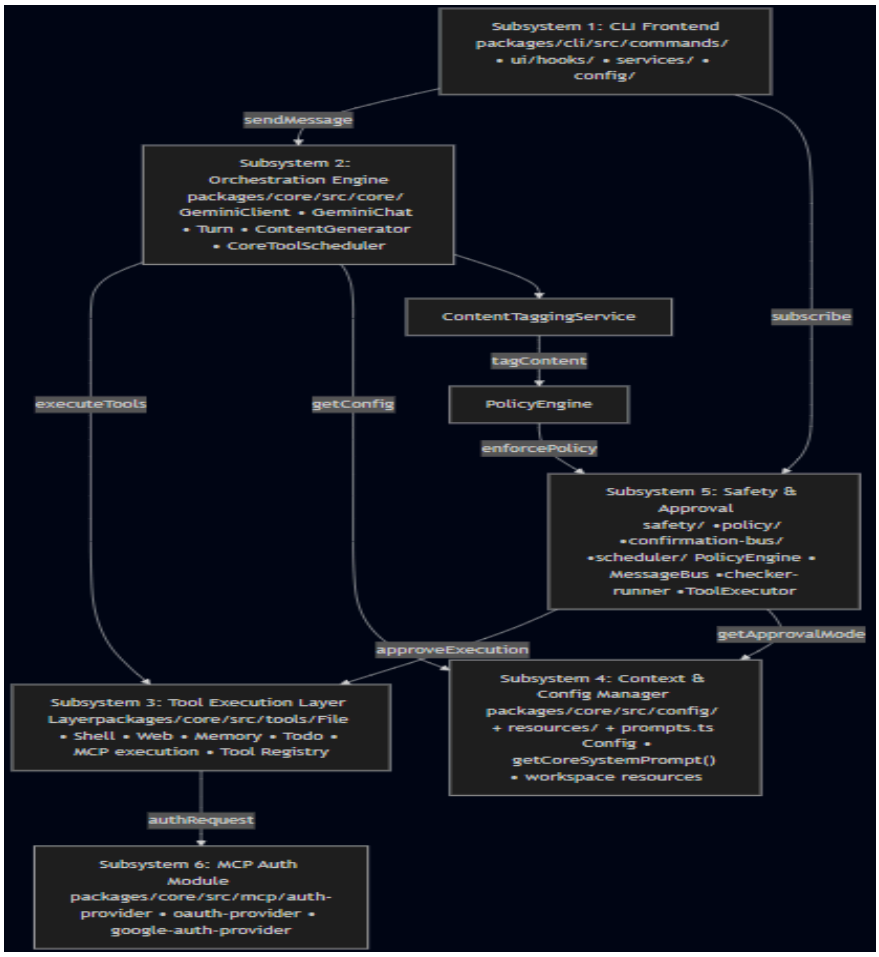
Interface Changes

- The InputHandler introduces a new method, tagContent(input), to invoke the ContentTaggingService.
- The InjectionService is extended with a method, such as processTaggedInput(input, tags), to enforce validation logic.
- The SandboxManager includes a method applyPolicy(tags) to dynamically adjust execution constraints.
- Internal APIs are updated to propagate content metadata alongside the input throughout the pipeline.

Impacted Files and Directories

- /security/ContentTaggingService -> new component for assignment content provenance tags
- /security/PolicyEngine -> new component for enforcing tag-based security policies
- /services/InjectionService -> modified to support tag-aware validation and filtering
- /sandbox/SandboxManager -> modified to apply dynamic sandbox policies based on tags
- /pipeline/InputHandler -> updated to integrate the tagging step into the input flow

These modifications directly reflect the high-level architectural enhancement by introducing a dedicated tagging and enforcement layer while extending existing security mechanisms. The design ensures minimal disruption to the current system while enabling more granular, context-aware security handling.



5 Use Cases and Sequence Diagrams

5.1 Use Case 1

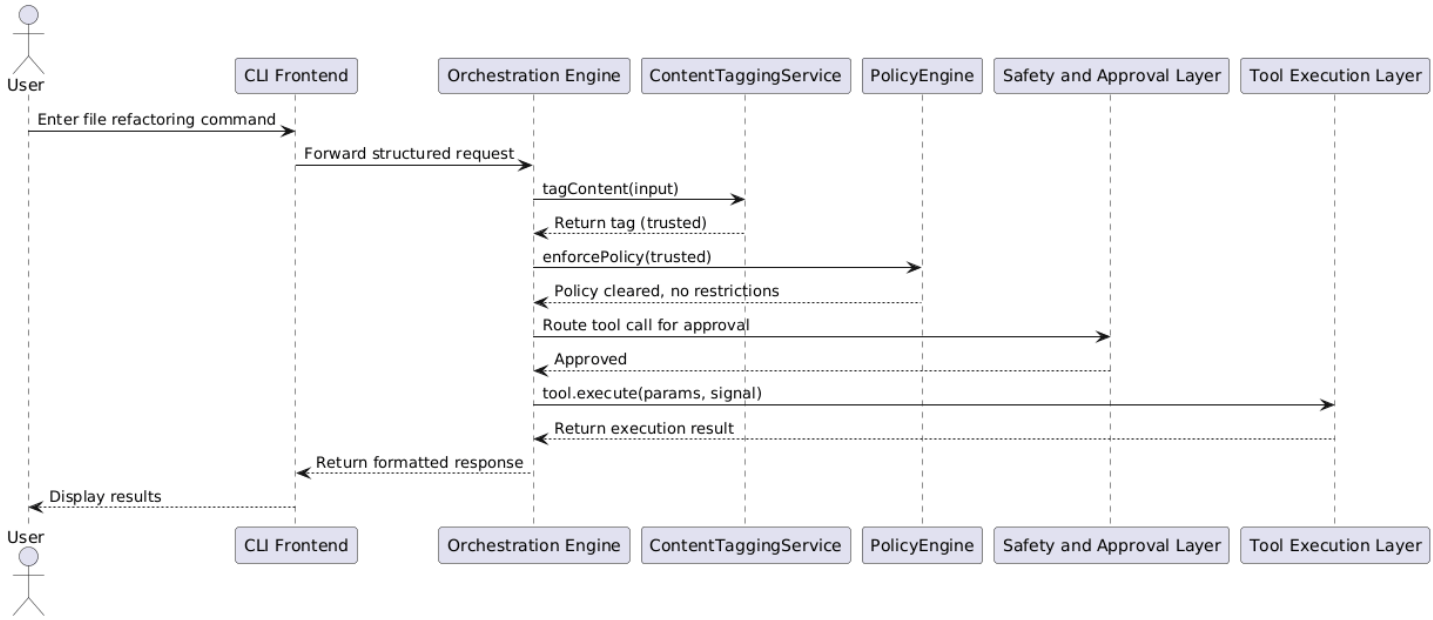
A primary use case for the proposed enhancement involves a developer using Gemini CLI to refactor code across multiple files. In this scenario, the user enters a command directly into the CLI, making it a trusted input source. The enhancement is designed to handle this case with minimal overhead, ensuring that legitimate workflows are not interrupted by the new security layer.

The flow begins when the user enters a file refactoring command into the CLI Frontend, which forwards a structured request to the Orchestration Engine. The Orchestration Engine passes the input to the new ContentTaggingService via tagContent(input), which analyzes the origin of the content and assigns it a trusted tag since it came directly from the user through the terminal. The tagged content is then passed to the

PolicyEngine via enforcePolicy(trusted), which confirms that no additional restrictions are required for trusted content and clears the request to proceed normally.

From this point, the flow continues through the existing architecture. The Orchestration Engine routes the tool call to the Safety and Approval Layer, which runs the existing approval pipeline and prompts the user for confirmation if needed. Once approved, the Tool Execution Layer executes the requested file operations via the tool.execute(params, signal) and returns the result back up through the Orchestration Engine to the CLI Frontend, which displays the final output to the user.

This use case demonstrates that the new tagging layer adds no friction to trusted workflows. The ContentTaggingService and PolicyEngine process the request quickly and pass it along without modification, preserving the existing user experience while still ensuring all content is tracked for provenance.



5.2 Use Case 2

A second use case demonstrates the enhancement of handling a potentially malicious scenario. In this case, a user performs a web search through Gemini CLI, and the external web content returned contains a prompt injection attempt. This is a known attack vector for LLM-based systems where malicious content retrieved from external sources attempts to override system instructions or manipulate model behaviour.

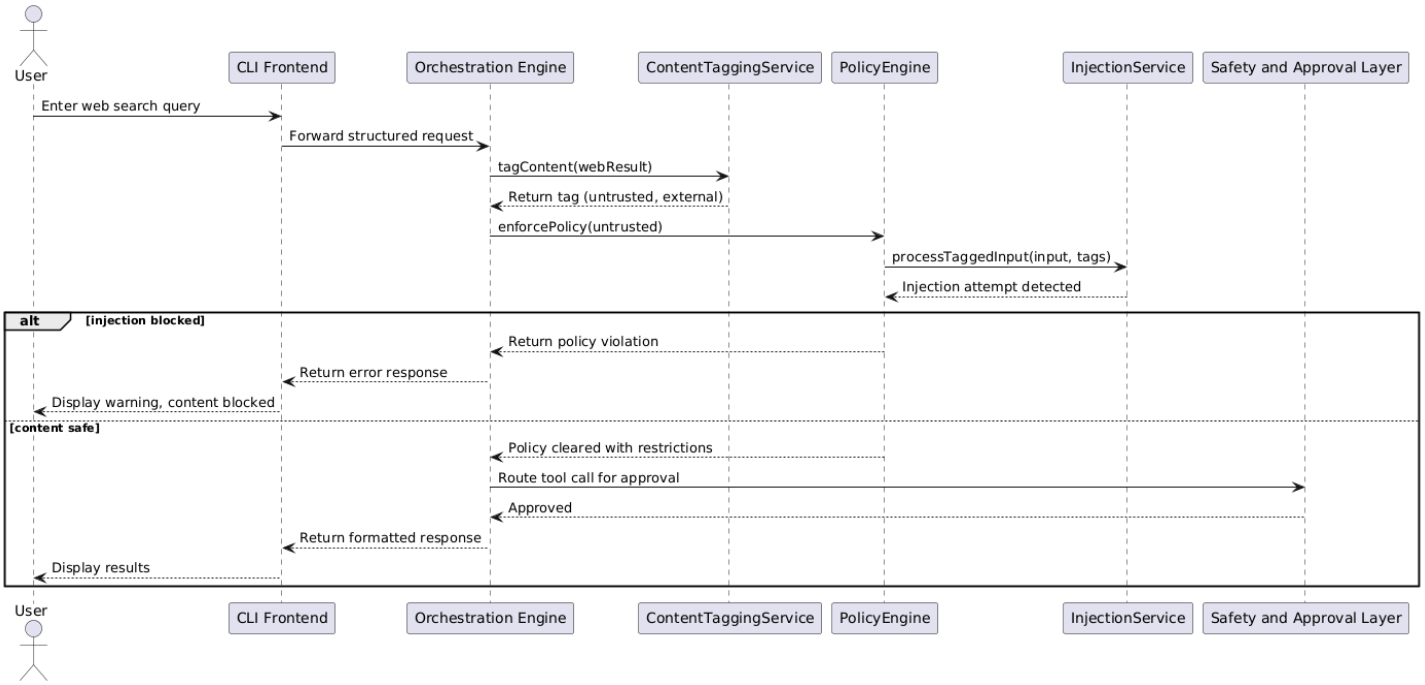
The flow begins when the user enters a web search query into the CLI Frontend, which forwards the request to the Orchestration Engine. The Orchestration Engine retrieves the web content and passes it to the ContentTaggingService via tagContent(webResult). Since the content originated from an external web source

rather than directly from the user or the system, the ContentTaggingService assigns it an untrusted and external tag and returns this to the Orchestration Engine.

The Orchestration Engine then passes the tagged content to the PolicyEngine via enforcePolicy(untrusted). Because the content is tagged as untrusted, the PolicyEngine applies stricter handling and routes the content to the InjectionService via processTaggedInput(input, tags). The InjectionService performs heightened sanitization and injection detection on the content and identifies a prompt injection attempt, returning a detection result to the PolicyEngine.

At this point, the flow branches. If an injection attempt is confirmed, the PolicyEngine returns a policy violation to the Orchestration Engine, which returns an error response to the CLI Frontend, and the user is shown a warning indicating the content was blocked. If the content passes the injection check despite being tagged as untrusted, the PolicyEngine clears it with restrictions and the request proceeds to the Safety and Approval Layer and onwards through the normal execution flow.

This use case demonstrates the core value of the proposed enhancement. Without the ContentTaggingService and PolicyEngine, the web content would have entered the pipeline without any provenance tracking, giving the injection attempt a path to influence model behaviour. With the new layer in place, external content is consistently identified, escalated for stricter validation, and blocked before it can cause harm.



6 Implementation Alternatives

6.1 First Alternative - Centralized Tagging and Policy Layer

This approach introduces a dedicated ContentTaggingService and PolicyEngine as a centralized layer early in the input pipeline. All incoming content is processed by this layer before being passed to downstream services like the InjectionService and SandboxManager.

In this setup, the ContentTaggingService assigns provenance-based metadata, such as trusted, untrusted, or external. It uses predefined rules based on source type, input channel, and retrieval origin. The PolicyEngine then applies consistent rules throughout the system. This ensures all components interpret and respond to tags in the same way.

The main advantage of this approach is the separation of concerns. Tagging logic and enforcement policies are separate from existing services, which improves maintainability. This allows for security updates without modifying core components. It also makes it easier to add new tag types or more sophisticated semantic analysis in the future.

However, this approach adds extra latency because of the additional processing step. It also requires updates to internal APIs to share metadata across the pipeline. Still, if implemented efficiently, the impact is expected to be minimal. Overall, this option offers a clear, scalable, and modular solution that fits well with the existing architecture.

6.2 Second Alternative - Distributed Tag-Aware Enhancement

Another option is to integrate tagging and policy logic directly into existing components like the InjectionService, InputHandler, and SandboxManager, instead of creating new standalone modules.

In this design, each component takes responsibility for identifying and managing untrusted content locally. For instance, the InjectionService could include logic to determine input origin, while the SandboxManager sets execution constraints based on context signals.

The advantage of this approach is lower architectural overhead, as it avoids adding new services and decreases the need for extra inter-component communication. It may also lead to slightly better performance due to fewer pipeline stages. However, this design creates tight coupling and repeated logic because multiple components must implement their own tagging and enforcement methods. This raises maintenance complexity and makes it harder to ensure consistent security behaviour throughout the system. It also limits scalability, as introducing new tagging strategies would require changes to multiple services.

As a result, while this approach may be easier to implement in the short term, it is less suitable for long-term growth and security strength.

7 Effects on System Quality Attributes

7.1 Maintainability

The centralized architecture of Alternative 1 significantly enhances the maintainability of security-related logic within Gemini CLI. Consolidating all provenance tagging logic into a single component,

ContentTaggingService (located in packages/core/src/tagging/), and all trust-based policy enforcement into the existing PolicyEngine (located in packages/core/src/policy/), ensures that future security updates remain localized. When new attack vectors necessitate modified risk heuristics or additional tag categories, updates are limited to ContentTaggingService, eliminating the need for parallel changes to InjectionService, SandboxManager, or other downstream components. This approach directly extends the separation of concerns principle embodied by the existing Safety and Approval Layer. The A2 reflection analysis confirmed that the safety pipeline resulted from a deliberate refactoring [6], dividing a monolithic safety block into four specialized modules to facilitate independent maintenance. ContentTaggingService maintains this architectural strategy. In contrast, Alternative 2 distributes tagging logic across multiple components, requiring coordinated changes across various files and subsystems, which increases both maintenance costs and security risks over time.

7.2 Evolvability

The structured tag envelope, comprising TRUST_LEVEL, SOURCE_TYPE, and RISK_SCORE, is designed to be extensible. New trust levels, source types, or scoring algorithms can be incorporated into ContentTaggingService without necessitating modifications to consuming components. The PolicyEngine's enforcement rules, already decoupled from the safety-checker implementation, as confirmed in the A2 architecture analysis [6], can be updated independently of the tagging logic. This modularity enables the security model to evolve incrementally, such as by supporting context-sensitive trust levels—where an MCP server is trusted for read operations but flagged for write operations—or by introducing embedding-based semantic intent classification as a future enhancement to ContentTaggingService, all without altering the Orchestration Engine, Tool Execution Layer, or CLI Frontend. In contrast, Alternative 2's tight coupling between tagging and component-level logic creates architectural obstacles to such progressive enhancements, as any upgrade to the tagging model would require replication or refactoring across multiple components, increasing the risk of behavioural inconsistency.

7.3 Testability

Alternative 1 markedly enhances the testability of Gemini CLI's security behaviour. ContentTaggingService features a well-defined input/output contract: given a content object with a specified source channel, it produces a deterministic tag envelope. Similarly, the PolicyEngine, implemented in packages/core/src/policy/policy-engine.ts, provides a testable contract by returning a deterministic enforcement decision for a given tag envelope with a known trust level. These characteristics enable both components to be unit-tested in isolation, without instantiating the entire pipeline. The explicit module boundaries that facilitated reasoning about the Safety and Approval Layer in the A2 analysis are preserved in this design. In contrast, Alternative 2 embeds tagging logic within existing component implementations, necessitating integration-level tests to verify tag behaviour. This approach is more costly, less reproducible in continuous integration environments, and more susceptible to brittleness when components are modified. Furthermore, incorrect tag decisions in Alternative 2 may not be attributable to a single component without comprehensive pipeline instrumentation.

7.4 Performance

The principal performance tradeoff introduced by the proposed enhancement is the addition of a synchronous tagging evaluation for each content object entering the pipeline. For direct user CLI input, which is typically tagged as TRUSTED, the overhead is minimal and consists of a source-channel lookup and a constant-time tag assignment. For external content, such as web fetch results or MCP tool responses, the RISK_SCORE computation may require lightweight pattern matching against known injection signatures, resulting in a small, bounded latency. This additional processing occurs only on content paths that already involve network I/O, which dominates total request time, rendering the tagging overhead negligible in practice. While Alternative 2 avoids centralized processing and may appear to reduce per-component overhead, the lack of centralization can lead to redundant trust signal evaluations by multiple downstream components, potentially resulting in greater overall overhead than Alternative 1's single shared tagging. In summary, the performance impact of Alternative 1 is minimal for trusted workflows and acceptable for external content, with the added latency justified by the corresponding security benefits.

8 SEI SAAM Analysis

With the above alternatives of implementing further security optimizations in mind, we will now perform SEI SAAM Analysis upon the overall solution NFR requirements, the stakeholders they affect, analyze the differences between the two alternatives against each NFR, and find the optimal alternative recommended for implementation.

8.1 Stakeholders

8.1.1 Security Engineers

The engineers in charge of actually implementing these changes will have the utmost responsibility for ensuring the changes are properly implemented, tested, and delivered. They will need to keep the system secure over time, and will also need to keep in mind how easy it is to audit, update, and extend the layer since it will need to constantly adapt to incoming, novel attack vectors/methods.

8.1.2 Enterprise Administrators

These are a special type of end user that will be in charge of distributing Gemini CLI across their enterprise and ensuring the product satisfies the company’s needs and security requirements. As such, they will have a high degree of interest placed upon policy configurability and compliance so that, for example, they are able to easily define what counts as untrusted in their company environments.

8.1.3 MCP Developers

Another type of end user that will work heavily on the customizability of Gemini CLI so that it fits their workflow needs. As third-party developers extending upon Gemini CLI, they will be monitoring whether or not the tagging layer breaks or complicates integrations and plugins they might create for their MCP calls.

8.1.4 General End Users

Finally, these are going to be the largest user base, housing general developers who just want a helping tool that is well-integrated into their home environments and provides general assistance. They will be interacting with the system daily and are some of the most vulnerable users to prompt injection attacks. While this vulnerability will be taken care of and maintained by Gemini CLI, they will also want to ensure that their workflows aren't interrupted or getting in their way in general due to the tool creating overhead or delays with security updates.

8.2 NFRs

With all of the affected stakeholders in mind, or at least the major ones, there are specific NFRs that pertain to particular stakeholder groups. A total of ten NFRs are identified and will be briefly mentioned and explained.

- **Security Engineers**
  - **Maintainability -** tagging logic should be easy to update independently
  - **Evolvability -** new trust levels or tag types should be addable without major refactoring
  - **Testability** - the tagging and enforcement logic should be unit-testable in isolation
- **Enterprise Administrators**
  - **Configurability** - policies should be customizable per deployment
  - **Auditability** - tag decisions and policy enforcement actions should be loggable
- **MCP Developers**
  - **Interoperability** - tags must propagate correctly through MCP tool calls
  - **Backward compatibility** - existing plugins shouldn't break with incoming Gemini CLI updates
- **General End Users**
  - **Performance** - latency added by tagging should be negligible and virtually unnoticeable
  - **Transparency** - they should understand when content is flagged
  - **Reliability** - false positives shouldn't block legitimate workflows

While the NFRs don't necessarily affect only a particular stakeholder group (e.g. the ones for General End Users also affect Enterprise Administrators greatly), they are most pertinent to the group they are attached to and are shown as such for easier interpretation of each NFR.

8.3 Evaluation

In the following table, we will evaluate the two alternatives presented in **Section 7** for implementing these security changes. They will be evaluated against the NFRs stated previously in **Section 8.2**, where if one alternative beats out the other, it will be highlighted in green to indicate superiority in that NFR.

Stakeholders	NFRs	Alt 1 - Centralized Tagging and Policy Layer	Alt 2 - Distributed Tag-Aware Enhancement
Security	**Maintainability**	Tagging logic is isolated in ContentTaggingService,	Logic might be duplicated across multiple components, updates

Engineers		allowing for updates to be modularized and easily maintainable without affecting other services.	require changes in multiple locations (InjectionService, Input Handler, SSandboxManager.
	Evolvability	New tags can simply be added to ContentTaggingService without updating other classes.	New tags will require modifying the numerous components that handle each tag independently.
	Testability	The added classes are isolated modules, thus easy to independently unit test	Tagging logic is entangled with component responsibilities, which will require integration-level tests
Enterprise Admins	**Configurability**	PolicyEngine acts as a centralized configuration point,t and policies can be customized for each deployment without touching other components.	No central policy location, which will render customization of deployments tedious by needing to modify each individual component separately
	Auditability	PolicyEngine, being centralized,d can log all tag decisions and enforcement actions in one place	Logging is possible, le but it is scattered across multiple components, making auditing or debugging more difficult to follow
MCP Developers	**Interoperability**	Tags have to be serialized and propagated through MCP tool calls, which requires API updates.	No new intercomponent pre- and post-conditions needed for tag serialization, as that is done within the components, thus existing MCP integrations are less likely to be disrupted.
	Backward Compatibility	Internal API changes are needed to carry metadata, and existing plugins might need some updates consistently	Extends existing components rather than introducing new ones, producing a lower risk of breaking existing plugins
General End Users	**Performance**	An additional pipeline stage (the new classes) could add some latency, but it will be minimal assuming proper optimization.	No extra pipeline stages
	Transparency	The central layer provides a single point where flagging events can be surfaced to the user consistently.	Flagging behaviour is distributed across components, which renders it harder to present consistent user-facing messages

	Reliability	Assuming a consistent execution within PolicyEngine, the risk of conflicting enforcement decisions is reduced significantly.	Independent tagging logic per component increases the risk of inconsistent behaviour (more risks and non-risks misidentified)

8.4 Recommendation

The team believes that **Alternative 1 - Centralized Tagging and Policy Layer** will be the more effective and recommended alternative to implement for the aforementioned security enhancements. It is substantially simpler to update for developers, maintains the high degree of security required for enterprise administrators, and has minimal effect on general end users. While it does come with a more difficult time for MCP Developers to adjust to changes, as they will feel breaking changes much more often than other stakeholders, this inconvenience will hopefully be mitigated with appropriate documentation, update notices, and anything else that will help these DIY users to continue DIY'ing their solutions. It additionally comes with a risk of higher latency, since additional security typically suffers from this, but with appropriate optimization from the engineers and developers, this will be a negligible issue.

9 Interactions with Existing Features

The proposed content tagging and policy enforcement layer is designed to integrate seamlessly with existing Gemini CLI security mechanisms without requiring major structural changes.

InjectionService

The InjectionService is extended to become tag-aware, allowing it to apply different validation strategies based on content provenance. For example, inputs tagged as *untrusted* or *external* can trigger stricter prompt injection detection, while trusted system instructions can bypass certain checks. This enhances existing injection defences without replacing them, making them more context-sensitive and effective.

SandboxManager

The SandboxManager leverages content tags to dynamically adjust execution constraints. For high-risk content, stricter sandboxing configurations (e.g., limited permissions or isolated environments) can be enforced automatically. This builds on the current sandboxing system by introducing adaptive security policies, rather than static configurations.

InputHandler

The InputHandler is modified to route all incoming content through the ContentTaggingService before entering the processing pipeline. This ensures that **no content bypasses provenance tracking**, maintaining consistency across the system.

Existing Injection Detection and Security Measures

Current mechanisms, such as prompt injection classifiers, markdown sanitization, and security prompt reinforcement remain unchanged but become **more effective when combined with tagging**. These systems can use tag metadata as an additional signal to improve detection accuracy.

MCP Tooling and External Integrations

To maintain interoperability, content tags are propagated through internal APIs and included in interactions with MCP tools. This ensures that third-party integrations can either respect or ignore tags without breaking functionality. Backward compatibility is preserved by making tag usage **optional and non-intrusive** for existing tools.

Overall Interaction Summary

Rather than replacing existing security features, the proposed enhancement acts as a **coordination layer** that improves how components communicate and make decisions about trust and risk. This results in a more consistent and context-aware security model while preserving the modular structure of the current system.

10 Testing Plan

Our testing strategy evaluates the correctness of the enhancement and its interaction with existing Gemini CLI components, in order to ensure security improvements do not degrade usability, performance, or compatibility.

Test Phase	Component / Scenario	Test Points	Validates
Unit Testing	**ContentTaggingService**	Verify correct tagging of inputs: · CLI input → trusted · Web/API content → untrusted, external · System-generated content → trusted Edge cases: mixed-origin inputs, malformed metadata	Correct provenance assignment per input source
	PolicyEngine	Validate correct enforcement decisions for each tag type Ensure correct routing: · trusted → minimal restrictions · untrusted → stricter validation via InjectionService Confirm policy configurability (enterprise overrides)	Accurate routing and enterprise configurability
	InjectionService (modified)	Verify tag-aware behaviour: · Stricter detection for untrusted inputs · No unnecessary blocking of trusted inputs	Tag-aware detection

			without over-blocking
	SandboxManager (modified)	Validate dynamic sandbox policy application based on tags Ensure correct isolation levels for high-risk inputs	Correct isolation applied per trust level
Integration Testing	**Scenario A — Trusted CLI Workflow**	Input: Direct CLI command (e.g., file refactor) Expected: · Tagged as trusted · No additional latency or restrictions · Normal execution through the Safety Layer and Tool Execution	No regression in user experience
	Scenario B — External Web Content	Input: Web-fetched content Expected: · Tagged as untrusted/external · Passed through InjectionService with stricter checks · Allowed execution if safe	Correct policy routing without false positives
	Scenario C — Prompt Injection Attempt	Input: Malicious web content (RAG poisoning) Expected: · Tagged as untrusted · InjectionService detects an attack · PolicyEngine blocks execution and returns a warning	Security effectiveness of the tagging layer
Interaction Testing	**Injection Classifiers & Sanitization**	Ensure tagging enhances detection (no redundancy/conflict) Validating combined signals improves accuracy	No conflicts with existing classifiers
	Sandboxing (gVisor / Containerization)	Confirm tag-based policies correctly adjust sandbox strictness Ensure no bypass of sandbox constraints	Sandbox constraints remain enforced
	MCP Integrations	Verify tag propagation through MCP APIs Ensure backward compatibility (untagged inputs still processed)	Backward-compatible MCP tag propagation
	Plan Mode / ReAct Loop	Confirm tags persist across multi-step reasoning loops Ensure malicious content cannot escalate privileges mid-loop	Tag persistence and privilege integrity

	File & Shell Execution	Validate high-risk inputs trigger restricted execution environments Ensure safe commands are not unnecessarily blocked	Correct execution environment applied per risk
Performance Testing	**Latency & Throughput**	Measure latency added by tagging layer: · Target: negligible (<5% overhead per request) Stress test with: · Large context inputs (1M token window) · High-frequency agent loops	No degradation in throughput, response time, or tool execution speed
10Security Testing	**Adversarial Testing**	Conduct adversarial testing: · Prompt injection (instruction override attempts) · Data exfiltration prompts · RAG poisoning with crafted documents Measure: detection rate, false positive rate, false negative rate	Resilience against known injection attack vectors
Regression Testing	**Existing Workflows**	Ensure all existing workflows remain functional: · CLI commands · Code generation · Tool execution Validate no breaking changes to: · APIs, plugins, existing automation scripts	No regressions introduced by the new layer
User Acceptance Testing (UAT)	**End-User Validation**	Validate: · Transparency of warnings · Minimal disruption to workflows · Clarity of error messages for blocked content	Acceptable user experience and trust in the system

11 Risks

Our proposed enhancement introduces several risks across security, performance, maintainability, and usability domains.

Category	Risk Item	Description	Potential Impact
Security	**False Negatives (Missed Attacks)**	Sophisticated prompt injections may bypass tagging or detection	System remains vulnerable despite the added layer

	False Positives (Overblocking)	Legitimate external content incorrectly flagged as malicious	Degraded user trust and workflow disruption
	Tag Spoofing / Misclassification	Incorrect tagging of content origin	Untrusted content treated as trusted
	Policy Misconfiguration	Enterprise-defined policies may be overly permissive or restrictive	Inconsistent or insecure behaviour across deployments
Performance	**Increased Latency**	Additional tagging and policy evaluation step in the pipeline	Slower response times, especially in agent loops
	Scalability Concerns	High-volume or large-context requests may amplify overhead	Degraded performance under heavy load
Maintainability	**Increased System Complexity**	New services (ContentTaggingService, PolicyEngine) add architectural overhead	Harder debugging and onboarding for new contributors
	API Changes	Metadata propagation requires interface updates across the pipeline	Increased maintenance burden and integration complexity
Compatibility	**MCP Integration Breakage**	Existing tools may not handle tag metadata correctly	Broken or inconsistent third-party integrations
	Backward Compatibility	Older scripts and plugins may not expect tagged inputs	Unexpected failures in legacy workflows
Reliability	**Inconsistent Tag Propagation**	Tags may be lost across pipeline stages or agent loops	Inconsistency in enforcement decisions
	Conflicting Enforcement Decisions	Different components may interpret tags differently	Unpredictable system behaviour
Usability	**Reduced User Experience**	Overly strict policies may interrupt normal workflows	User frustration and reduced adoption

	Lack of Transparency	Users may not understand why content is blocked	Confusion and mistrust in the system
Operational	**Policy Management Overhead**	Enterprises must define and maintain security policies	Increased operational burden for administrators
	Logging & Storage Overhead	Provenance tracking increases metadata storage requirements	Higher infrastructure costs

12 Lessons Learned

Through this architectural enhancement, we gained a better understanding of how to design and integrate new functionality into an existing system. One key lesson was the importance of properly scoping an enhancement. Our initial idea around security was too broad, but through refinement, we translated it into concrete components, namely the ContentTaggingService and PolicyEngine. This showed the importance of turning high-level concepts into implementable architectural solutions.

We also learned how to extend existing systems rather than redesigning them. Our enhancement was integrated as a modular layer that works alongside components such as the InjectionService and SandboxManager, reinforcing loose coupling and maintainability.

Finally, evaluating different implementation approaches helped us understand architectural trade-offs, particularly between modularity, performance, and complexity. Overall, this assignment improved our ability to design practical, well-justified architectural enhancements.

13 Conclusion

This report proposed an architectural enhancement to Gemini CLI that addresses a key security issue: the lack of consistent tracking of content origin throughout the system. While existing protections like injection detection and sandboxing are effective, they do not distinguish between trusted and untrusted inputs, leaving the system vulnerable to prompt injection attacks.

To address this, a Content Tagging and Policy Enforcement layer were added which classifies content based on its source and trust level. This allows components to make context-aware decisions, applying stricter controls to untrusted inputs while preserving efficiency for trusted workflows.

Overall, the centralized tagging approach provides a scalable and maintainable solution with minimal performance impact. This enhancement strengthens system security and establishes a foundation for more robust and reliable behavior in future LLM-based systems

14 AI Collaboration Report

As usual, we have kept using Claude's Sonnet Version 4.6 as our trustworthy AI teammate, for the same reasons outlined in A1 and A2 (Claude models' superiority in code and architectural understanding compared to other models). This time, we very well could have pivoted to another model, such as ChatGPT's current free-tier models, as, generally, ChatGPT is very effective at brainstorming (as outlined in A1), and that was our anticipated main use case upon first review of the assignment requirements. However, we remained with Sonnet because of our built-up knowledge of how to effectively prompt Sonnet for our use cases and how to interpret the responses from Sonnet. Additionally, through other uses of ChatGPT, we have realized that it tends to ramble a lot and is quite verbose with its responses, albeit with the upside that it provides a good bit of detail. Nonetheless, Sonnet also provides a good bit of detail and actually provides good, thoughtful suggestions or inclusions (at times, citations) alongside exactly what it is responding to without the need to read through large chunks of text, which would be required to do with ChatGPT's responses.

Since the group required to plan and assign tasks definitively before any feedback could be received from A2 in any capacity (including the group feedback) due to heavy weeks ahead for all group members' schoolwork schedules, we had to omit Sonnet's consolidating feedback and providing tips for writing this report out and presenting the presentation. As such, we mainly stuck with brainstorming ideas and document outlining.

14.1 Addition Brainstorming

Through group discussions, we had an initial consensus that many, if not all, of the chatbot services nowadays have security risks prone to them. More specifically, one of our group members who unofficially researched this topic in some quantity noted that LLMs are susceptible to prompt injection attacks and lack Personal Identifiable Information (PII) security. Thus, we agreed that this was an avenue we would like to explore, and asked Sonnet to provide its insights as to whether these risks are present within Gemini CLI. It found that there are some measures in place, along with some third-party plugins that aid with the above to an extent. However, it also identified known vulnerabilities that are present in the code, including undetectable arbitrary code execution by using Markdown code blocks within a `README.md` file, an instance where cyberattackers extracted sensitive corporate data by embedding hidden instructions in a shared Google Doc or calendar invite [5], and finally through Google DeepMind's own research found that undefended Gemini CLI systems were at risk and that advanced TAP methods executed their attacks properly almost 100% of the time [2], [3]. After validating these topics, we were happy to proceed and asked it to formulate a clear explanation of what opportunities exist for our group in terms of creating this solution by addressing the gaps of semantic vulnerability and inconsistent I/O separation at the architectural level, leaving Gemini CLI prone to attacks [4]. We then submitted these opportunities to our TA for approval and proceeded with elaborating on our points throughout the report.

14.2 Planning Document Outline

Similarly to A1 and A2, we gave the project description from the detailed description and the rubric and let it come up with the necessary sections for this report, which we then adjusted as necessary. Once we had an idea

of what to produce an enhancement for, and prior to getting approval from our TA, we did some quick analysis and discussion in a meeting of what to write for each section, given the overall idea. It also helped create the Table of Contents page, so that we can eliminate some tedious manual work that would come from it.

14.3 AI Collaboration Methodology

As mentioned earlier, Sonnet was utilized mainly just for brainstorming at this time, while we handled the writing and conceptualization of the overall solution post-brainstorming. The overall document outline was consistent with A1 feedback and how we wrote A2, and we adjusted anything if necessary. For the brainstorming portion, we did quick cross-referencing with a quick search on any of the information that Sonnet had provided as proof or reasoning behind its points (anything that was referenced in the earlier paragraphs of **Section 14**) to make sure that it wasn't hallucinating. After inspection, either proved positive or negative (usually, it was positive and found that the reasoning was grounded), we proceeded with elaborating on those brainstorming points and with any next steps we thought were appropriate. For example, after validating that Gemini CLI had the flaws mentioned, we then asked for what exact opportunity we had with an enhancement to complete what we needed.

We estimate that our AI teammate did less work in A3, which is less work than A1 and A2. This especially comes from us not needing it to produce diagrams or revise entire architectures anymore, and we really only needed ideas generated from Sonnet this time. Any diagrams we were already capable of creating ourselves since they simply built off of prior diagrams (e.g. extending or modifying parts of the A2 updated conceptual architecture to fit our new changes), or were diagrams we were already highly familiar with creating (e.g. sequence diagrams). Thus, we give about ~10% of the group efforts this time around, for help with ideation and outlining.

Sonnet has been an incredibly high-quality teammate, and our overall usage of this model has been shaped and changed throughout the past few weeks of working on this project. Sonnet has proved highly useful and saved tremendous amounts of time that we very much needed in order to keep sane and to not fall behind on work for this project alongside our other scholarly or personal commitments. We enjoyed the effectiveness brought from utilizing Sonnet's strengths, and learned how to better have Sonnet aid us by prompting exactly what was needed for our end, and confirming anything necessary to ensure over-reliance was not going to be something our group fell into.

15 References

[1] OWASP Gen AI Security Project, "LLM01:2025 Prompt Injection," OWASP Top 10 for Large Language Model Applications, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[2] Google DeepMind Security and Privacy Research Team, "Lessons from Defending Gemini Against Indirect Prompt Injections," Google DeepMind, May 2025. [Online]. Available: https://storage.googleapis.com/deepmind-media/Security%20and%20Privacy/Gemini_Security_Paper.pdf

[3] Google DeepMind, "Advancing Gemini's Security Safeguards," Google DeepMind Blog, May 2025. [Online]. Available: https://deepmind.google/blog/advancing-geminis-security-safeguards/

[4] S. Cox, "Code Execution Through Deception: Gemini AI CLI Hijack," Tracebit Blog, Jul. 28, 2025. [Online]. Available: https://tracebit.com/blog/code-exec-deception-gemini-ai-cli-hijack

[5] Noma Labs, "GeminiJack: Indirect Prompt Injection in Google Gemini Enterprise and Vertex AI Search," Noma Security, Feb. 2026. [Online]. Available: https://noma.security/noma-labs/geminijack/

[6] Group 5 — It Compiles (Sometimes), "A2 - Concrete Architecture Report - It Compiles (Sometimes)," Queen's University CISC 322/326, 2026. [Online]. Available: https://spy772.github.io/It-Compiles-Sometimes/

[7] Anthropic, "Claude," Claude.ai, 2026. [Online]. Available: https://claude.ai/